Jordan Phillips

HW1: Language Models from Scratch (Part 1)

1. QUESTION AND HYPOTHESIS

This assignment investigates the differences between recurrent neural networks and Transformers for character-level language modeling. It also examines how context length affects model performance and whether Transformer attention heads learn distinct patterns. I hypothesized that vanilla RNNs would perform worse than LSTMs and GRUs because they have more difficulty learning long-range dependencies. I also expected larger models to perform better, Transformers to use longer context more effectively, and different attention heads to develop specialized behaviors.

2. EXPERIMENTAL SETUP

All experiments used the Tiny Shakespeare dataset. Character-level tokenization was used for the architecture, context-length, and attention experiments. Character and byte tokenizers were compared in the untrained-model experiment. Experiments were run on a MacBook Air using PyTorch on CPU with FP32 precision. MPS was not used because of a runtime compatibility issue. The architecture and context-length experiments used approximately 9.83 million training tokens per run. The architecture comparison used 1,200 steps, batch size 32, and sequence length 256. The context-length experiment adjusted the number of steps to maintain the same token budget across configurations. Two random seeds (1337 and 42) were used for the architecture comparison. The context-length and attention experiments used seed 1337. Approximate computational cost was estimated using FLOPs ≈ 6ND, where N is the number of parameters and D is the number of training tokens. The architecture experiment took approximately 139.1 minutes across all 12 runs. The context-length experiment took approximately 105.4 minutes across all 8 runs.

Git commit: 97275deaa9bb39be1041325401cf8002fe02fc32

3. RESULTS

3.1 Metrics by Hand

For the trained character-level LSTM (lstm_s1337), the recorded validation metrics were loss = 1.558529, validation tokens = 111,360, validation bytes = 111,360, perplexity = 4.751826, and BPB = 2.248482.

The metrics were independently verified using the following calculations:

Perplexity = exp(loss) = exp(1.558529) = 4.751826.

Bits/token = loss / ln(2) = 1.558529 / ln(2) = 2.248482.

Bits/byte = bits/token × validation tokens / validation bytes = 2.248482 × 111360 / 111360 = 2.248482.

The calculated values match the logged metrics. Bits per token and bits per byte are equal because the character tokenizer represents one byte per token for this dataset. I also evaluated models using character and byte tokenizers after one training step. The character tokenizer had a vocabulary size of 65 and a validation loss of 4.1601, compared with ln(65) = 4.1744. The byte tokenizer had a vocabulary size of 256 and a validation loss of 5.4436, compared with ln(256) =

5.5452. Both measured losses were close to their theoretical uniform-prediction losses. The byte tokenizer had a higher initial loss because its larger vocabulary increases the uncertainty of predicting the next token. Small differences from ln(V) are expected because the models received one training step and their predictions were not perfectly uniform.

3.2 RNN vs. LSTM vs. GRU vs. Transformer

All architectures received approximately 9.83 million training tokens per run and were evaluated using two random seeds.

Architecture | Parameters (M) | Mean BPB | Standard deviation
RNN | 0.2803 | 2.7650 | 0.0058
GRU | 0.8067 | 2.2898 | 0.0040
LSTM | 1.0698 | 2.2485 | 0.0000*
LSTM 512 | 4.2368 | 2.1165 | 0.0065
Textbook GPT | 3.4383 | 2.3809 | 0.0126
Modern GPT | 3.1626 | 2.1505 | 0.0070

*The LSTM standard deviation is rounded to four decimal places.

Figure 1 (left): Architecture learning curves (validation BPB versus training tokens).

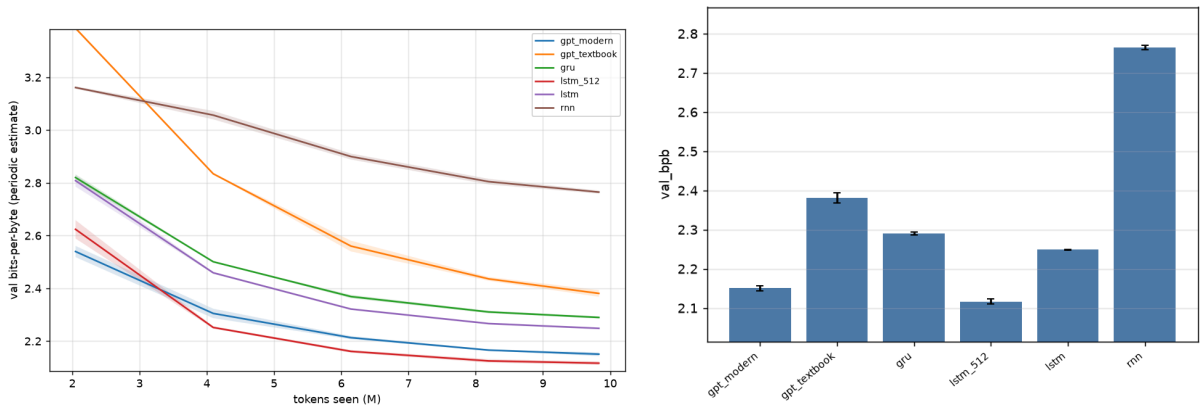


Figure 2 (right): Final validation BPB by architecture.

The vanilla RNN performed substantially worse than the other architectures, achieving 2.7650 BPB. The GRU and LSTM performed considerably better. This is consistent with the ability of gated recurrent architectures to regulate information flow and preserve useful information over longer sequences, reducing the effects of vanishing gradients.

The larger LSTM achieved the best overall result, with 2.1165 BPB compared with 2.2485 for the smaller LSTM. This improvement came at the cost of approximately four times as many parameters and considerably longer training time.

The modern Transformer achieved 2.1505 BPB, outperforming the textbook Transformer at 2.3809. The modern model combines RMSNorm, rotary positional embeddings, SwiGLU, and removal of linear biases. Since these modifications were introduced together, their individual contributions cannot be isolated.

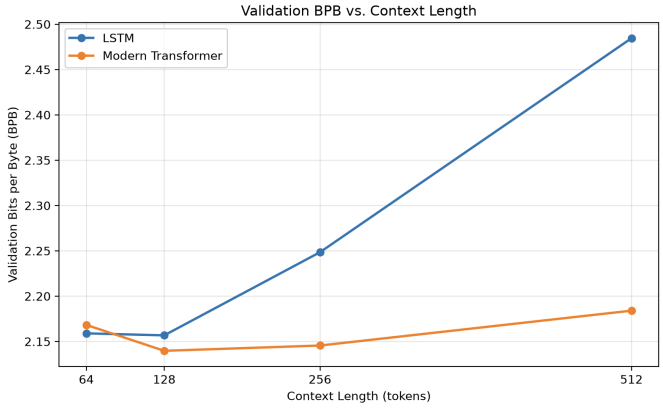
Paired statistical comparisons were performed using the two shared random seeds (1337 and 42). The GRU improved over the vanilla RNN by 0.4752 BPB ($p = 0.0018$), while the LSTM improved over the GRU by 0.0413 BPB ($p = 0.0437$). Increasing the LSTM hidden dimension to 512 improved BPB by 0.1320 ($p = 0.0220$). The modern Transformer improved over the textbook Transformer by 0.2304 BPB ($p = 0.0109$).

3.3 Context Length

LSTM and modern Transformer models were evaluated with context lengths of 64, 128, 256, and 512 tokens. Each run received approximately 9.83 million training tokens.

Context length | LSTM BPB | Modern Transformer BPB
64 | 2.1590 | 2.1683
128 | 2.1568 | 2.1397
256 | 2.2485 | 2.1455
512 | 2.4845 | 2.1840

Figure 3: Validation BPB versus context length.



The LSTM achieved its best result at a context length of 128, with nearly identical performance at 64. Its performance deteriorated at 256 and 512. This suggests that longer sequences did not provide useful additional information under the fixed training budget and may have made optimization more difficult.

The Transformer also performed best at 128. Its performance remained relatively stable at 256 but declined slightly at 512. Compared with the LSTM, the Transformer was more robust to increasing context length. These results suggest that the Transformer used longer context more effectively, although neither architecture benefited from increasing context beyond 128 in this experiment.

3.4 Attention Analysis

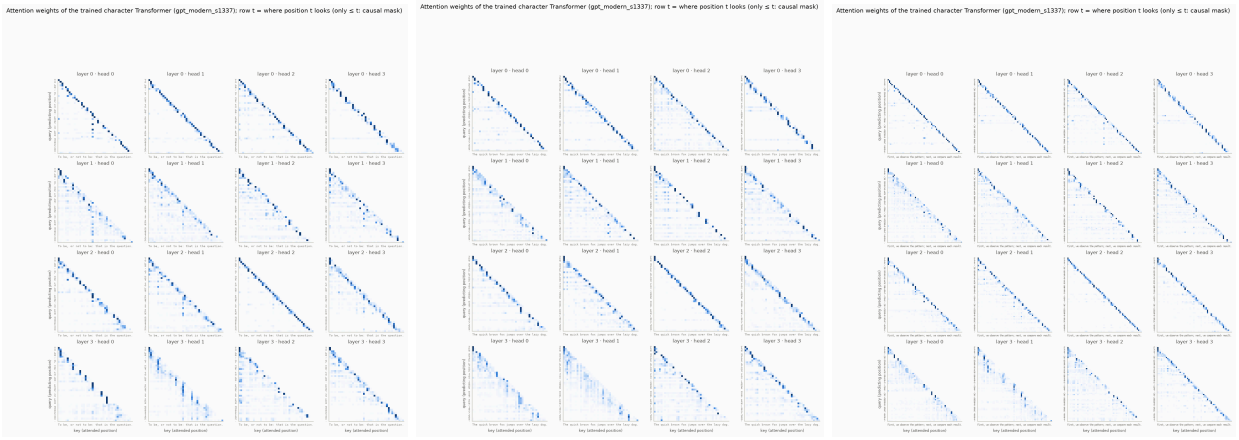
Attention maps were generated using the trained modern Transformer from the architecture experiment. Three text examples were evaluated: "To be, or not to be: that is the question.", "First, we observe the pattern; next, we compare each result.", and "The quick brown fox jumps over the lazy dog."

The model contains four layers with four attention heads per layer, giving 16 heads in total. Numerical analysis identified two notable attention patterns.

Layer 2, Head 2 exhibited strong previous-token attention, assigning an average weight of 0.6333 to the immediately preceding character. This was the highest previous-token attention value among the heads and corresponded to a strong diagonal pattern in the visualization.

Layer 1, Head 3 exhibited the strongest word-start attention, assigning an average total attention mass of 0.3736 to positions corresponding to the beginnings of words.

These results suggest that different attention heads learn distinct behaviors. Some focus heavily on nearby characters, while others distribute attention toward positions associated with word boundaries.



Figures 4, 5, 6: Attention maps for the modern Transformer on three text inputs: (left) Shakespeare excerpt, (center) structured prose, and (right) a simple English sentence. Brighter regions indicate stronger attention weights.

4. ANALYSIS

The experiments demonstrate several differences between recurrent architectures and Transformers. The vanilla RNN's substantially worse performance is consistent with its difficulty preserving information across long sequences. LSTMs and GRUs use gating mechanisms to regulate information flow and retain useful information. Increasing LSTM model size improved performance in the architecture experiment. The larger LSTM achieved the lowest BPB overall, although this improvement required greater computational resources.

The context-length experiment showed that longer context does not necessarily improve performance. The LSTM deteriorated substantially at longer context lengths, while the Transformer remained more stable. This supports the hypothesis that Transformers make more effective use of longer context, although both architectures experienced diminishing returns. The negative context-length results are not necessarily bugs. All configurations completed successfully, used the same training-token budget, and produced validation metrics. However, only one seed was used, so the results cannot establish that these differences would consistently occur across random initializations.

5. CONCLUSION AND LIMITATIONS

The experiments showed that gated recurrent architectures and Transformers substantially outperform vanilla RNNs on character-level language modeling. The larger LSTM achieved the best overall BPB, while the modern Transformer outperformed the textbook Transformer. The

context-length experiment showed that increasing context beyond 128 tokens did not improve performance for either architecture. The Transformer was more robust to longer context lengths than the LSTM. Attention analysis also demonstrated distinct previous-token and word-start patterns across Transformer heads.

The main limitations are the small number of seeds, particularly the single seed used for context-length comparisons, and the limited size of the Tiny Shakespeare dataset. The paired statistical tests used only two seeds, giving them limited statistical power. Although all four planned architecture comparisons had nominal p-values below 0.05, only the RNN-to-GRU and textbook-to-modern Transformer comparisons remained below the Bonferroni-adjusted threshold of 0.0125 for four comparisons. Additionally, the LSTM and Transformer used different weight-decay settings in the context-length experiment, limiting direct comparisons of their absolute performance. The modern Transformer also combines several architectural modifications, preventing attribution of improvements to individual changes.

APPENDIX: REPRODUCIBILITY

Experiment configurations:

- nanolm/configs/sweeps/01_char_models.yaml
- nanolm/configs/sweeps/context_length.yaml
- nanolm/configs/laptop/L3_tokenizer.yaml

Experiment summaries:

- outputs/nanolm/sweeps/char_models/summary.csv
- outputs/nanolm/sweeps/context_length/summary.csv

Additional scripts:

- plot_context_length.py
- analyze_attention.py

Figures:

- [1] char_models/curves.png
- [2] char_models/bars.png
- [3] context_length/context_length_bpb.png
- [4] nanolm_figs/attention_shakespeare.png
- [5] nanolm_figs/attention_sentence.png
- [6] nanolm_figs/attention_prose.png